

HelixCode

HelixCode

La plateforme de développement distribué AI qui divise le travail, le préserve et ne perd jamais votre progression.

Résumé

HelixCode est une plateforme de développement distribué AI de niveau entreprise, basée sur Go, qui répartit les tâches de développement en segments intelligents à travers un réseau de travailleurs gérés par SSH. Elle intègre un système de points de contrôle et de restauration automatique pour garantir qu'aucune avancée n'est jamais perdue. Elle unifie l'intégration multi-fournisseurs LLM, les workflows couvrant l'ensemble du cycle de développement, et la livraison multiplateforme, le tout accessible via les interfaces REST, CLI, TUI et MCP.

Description courte

HelixCode est une plateforme de développement distribué AI écrite en Go. Elle divise le travail en tâches intelligentes réparties sur un réseau de travailleurs SSH, préserve la progression grâce à des points de contrôle et des restaurations automatiques, intègre plusieurs fournisseurs LLM et pilote l'ensemble du cycle de développement via les interfaces REST, CLI, TUI et MCP.

Description détaillée

HelixCode est une plateforme de développement distribué AI de niveau entreprise (`dev.helix.code`, licence MIT) construite autour d'une promesse simple, littéralement exprimée par son slogan : diviser le travail, le préserver et ne jamais perdre sa progression. Conçue pour une répartition intelligente des tâches, la sauvegarde automatique des avancées et des workflows de développement multiplateformes, elle est écrite en Go pour répondre aux exigences de concurrence et de portabilité en un

seul binaire que requiert l'informatique distribuée. Elle intègre nativement la création de points de contrôle, la restauration et la surveillance en temps réel, plutôt que comme des fonctionnalités optionnelles.

Son architecture superpose une interface API combinant REST, WebSocket et MCP à un ensemble de services centraux spécialisés : authentification JWT et gestion des sessions, gestion de pools de travailleurs basée sur SSH avec surveillance de l'état de santé, gestion des tâches avec points de contrôle et traitement des dépendances, gestion des projets et des workflows, ainsi qu'une couche unifiée de fournisseurs LLM. Toutes ces données sont persistées sur PostgreSQL, avec Redis disponible en option comme niveau de coordination et de mise en cache. Les travailleurs distribués s'installent automatiquement sur un réseau, ce qui permet de dimensionner le parc en pointant simplement le serveur vers une machine, sans configuration manuelle. Les interfaces multi-clients couvrent CLI, l'interface en terminal, REST et les frameworks mobiles, rendant la plateforme accessible aussi bien depuis un script, un terminal ou une application.

HelixCode prend en charge l'intégralité du cycle de développement de bout en bout : planification, construction, tests et refactoring s'exécutent automatiquement avec une gestion des dépendances et un suivi du contexte multi-session, permettant à un projet de longue haleine de conserver sa cohérence malgré les interruptions et les changements de machine. Elle intègre plusieurs fournisseurs LLM — llama.cpp, Ollama et OpenAI — derrière une seule interface, tout en ajoutant une sélection de modèles adaptée au matériel, capable de détecter les ressources disponibles (CPU, GPU, mémoire) et d'ajuster le modèle en conséquence. Elle supporte également des stratégies de raisonnement avancées comme le *chain-of-thought* et le *tree-of-thoughts* pour les problèmes nécessitant plus qu'une seule passe. Le protocole Model Context Protocol est implémenté sur plusieurs transports pour un échange standardisé d'outils et de contextes, et des notifications multi-canaux (Slack, Discord, Email, Telegram) informent les équipes de l'avancement des travaux distribués. La plateforme cible Linux, macOS, Windows, Aurora OS et SymphonyOS.

Contenu

Pourquoi nous l'avons conçu

Le développement distribué et assisté par AI perd généralement en contexte et en progression lorsque les tâches sont réparties entre plusieurs machines ou interrompues. HelixCode a été conçu pour rendre la division des tâches intelligente et la sauvegarde du travail automatique - afin qu'un effort de développement

d'envergure puisse être décomposé, distribué sur un réseau de travailleurs, sauvegardé à des points de contrôle, puis repris ou annulé sans perte d'état.

Pourquoi c'est révolutionnaire

Il rend le développement distribué assisté par AI *résilient* - une capacité qui n'était jamais réaliste lorsque les équipes assemblaient ces éléments manuellement. Trois fonctionnalités habituellement dispersées dans trois outils distincts ne font plus qu'un : le calcul distribué (réseaux de travailleurs SSH avec installation automatique et surveillance de l'état), l'assistance au développement AI (LLM multi-fournisseurs dotés de raisonnement et d'appel d'outils), et l'automatisation des workflows sur l'ensemble du cycle de vie. Le liant, c'est la sauvegarde des points de contrôle basée sur une base de données : comme l'état des tâches, les sauvegardes et les dépendances sont persistés dans PostgreSQL, un travail s'étalant sur plusieurs machines et sessions peut être annulé ou repris exactement là où il s'est arrêté. Les interruptions et la répartition du travail cessent d'être une source de perte de progression pour devenir un événement routinier et récupérable.

Ce qui est innovant

- La préservation du travail comme primitive fondamentale : sauvegarde et annulation automatiques appliquées aux tâches de développement *distribuées*, afin que la progression survive aux interruptions et aux pannes matérielles au lieu de s'évanouir avec elles.
- Sélection de modèles adaptée au matériel, qui analyse le CPU/GPU/mémoire détectés et associe chaque tâche à un modèle que la machine peut exécuter efficacement - sans réglage manuel par travailleur.
- Une plateforme, cinq interfaces : REST, WebSocket, CLI, TUI et MCP, MCP étant lui-même accessible via plusieurs protocoles pour permettre aux outils et agents de s'intégrer quelle que soit leur méthode de connexion.
- Une portée multiplateforme qui dépasse le trio habituel des postes de travail pour inclure Aurora OS et SymphonyOS, élargissant ainsi le parc de travailleurs à des plateformes souvent ignorées par la plupart des outils.

Principaux défis techniques et nos solutions

- **Ne pas perdre de travail dans des tâches distribuées et interruptibles.** Lorsqu'un travail est réparti entre plusieurs machines, tout plantage ou interruption laisse généralement en plan ce qui était en cours. Nous avons modélisé la tâche elle-même comme porteuse de points de contrôle et de dépendances, persistés dans PostgreSQL, afin que le système puisse revenir au dernier état valide ou reprendre à partir de celui-ci - une résilience ancrée dans la couche de données plutôt que dans un état en mémoire fragile.
- **Gérer un parc de travailleurs hétérogène.** Un réseau de machines Linux, macOS, Windows, Aurora et SymphonyOS est une cible mouvante en termes de disponibilité et de configuration. Nous le gérons grâce à un service dédié de pool de travailleurs qui assure l'enregistrement basé sur SSH, l'installation automatique sur les nouveaux nœuds et une surveillance continue de l'état, maintenant ainsi le parc connu et maîtrisé à mesure que les machines apparaissent et disparaissent.
- **Hétérogénéité des fournisseurs et du matériel.** Les backends LLM et les machines qui les exécutent varient considérablement en termes de capacités. Nous avons masqué cette complexité derrière une interface unifiée de fournisseur LLM, couplée à une détection matérielle (CPU/GPU/mémoire) qui pilote une sélection intelligente des modèles, garantissant que le bon modèle s'exécute sur la bonne machine sans que l'appelant ait à se soucier de l'un ou de l'autre.

Contenu

Pile technologique

- **Go (1.26+ module interne)** — choisi pour sa concurrence basée sur les goroutines et sa sortie en binaire unique, exactement ce dont a besoin un système de workers distribués : un parallélisme économique pour l'orchestration et un binaire autonome qui s'installe automatiquement sur n'importe quel nœud. Il embarque tous les services principaux ainsi que les binaires CLI/server.
- **Gin (framework HTTP)** — retenu pour sa couche REST rapide et minimaliste, à faible surcharge ; il sert la surface /api/v1 (authentification, workers, tâches, projets) à laquelle tous les clients s'adressent.
- **PostgreSQL 15+ (via pgx/v5)** — sélectionné comme système de référence durable, car la sauvegarde des points de contrôle et le retour arrière exigent une persistance transactionnelle ; il héberge le schéma de calcul distribué à 11 tables (utilisateurs,

workers, tâches, projets, sessions, fournisseurs_llm, notifications) qui rend possible la préservation du travail.

- **Redis 7+ (optionnel, go-redis/v9)** — adopté comme couche de mise en cache et de coordination facultative, accélérant les chemins critiques sans devenir une dépendance incontournable, de sorte qu'un déploiement minimal fonctionne avec PostgreSQL seul.
- **SSH** — choisi comme protocole de transport pour le contrôle des workers précisément parce qu'il est déjà omniprésent et sécurisé ; il gère l'enregistrement des workers, leur installation automatique et l'exécution de commandes à distance sur l'ensemble du pool, sans nécessiter le déploiement préalable d'un agent spécifique.
- **Model Context Protocol (MCP)** — retenu pour l'échange standardisé d'outils et de contextes, permettant aux outils et agents externes de s'intégrer via un protocole ouvert ; implémenté avec une prise en charge multi-transport pour s'adapter aux clients, quel que soit leur point de connexion.
- **Fournisseurs LLM (llama.cpp, Ollama, OpenAI)** — sélectionnés pour couvrir à la fois l'inférence locale et hébergée derrière une interface unifiée, permettant une sélection adaptée au matériel qui achemine une tâche vers un modèle local ou hébergé sans que l'appelant en ait conscience.

Statut et notes de transparence

- **Statut : bêta.** Le README annonce un état « COMPLÈTEMENT TERMINÉ / les 5 phases achevées » ; cette complétude est déclarée par le projet et non vérifiée de manière indépendante, aussi la page la considère-t-elle comme bêta.
- Toutes les précisions ci-dessus proviennent du README du dépôt ; les formulations marketing (slogans) relèvent de l'éditorial et non de métriques issues de la source.

Niveau de priorité : Helix-primaire.